

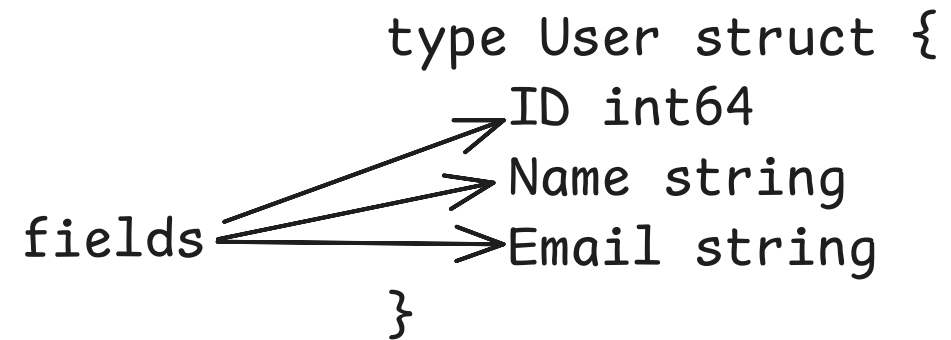
Lecture 04 - Structs and Methods

1 What is a struct?

Typed collection of fields useful for grouping data together to form records

```
type User struct {  
    ID int64  
    Name string  
    Email string  
}
```

fields



```
user := User{  
    ID: 1,  
    Name: "Almas",  
    Email: "almas@example.com",  
}
```

```
user.ID  
user.Name
```

```
user.ID = 3
```

```
fmt.Print
```

2 Struct fields

3

Creating struct values

clearest approach

```
user := User{  
    ID: 1,  
    Name: "Almas",  
    Email: "almas@example.com",  
}
```

```
user := User{  
    Name: "Almas",  
}
```

User 0x0004



ID: 0
Name: Almas
Email: ""

positional initialization

```
user := User{1, "Almas", "almas@example.com"}
```

```
user1 := User{
    Name string,
}

user2 := user1

user2.Name = Maksat

fmt.Println(user1.Name) // Almas
```

4 Structs use value semantics

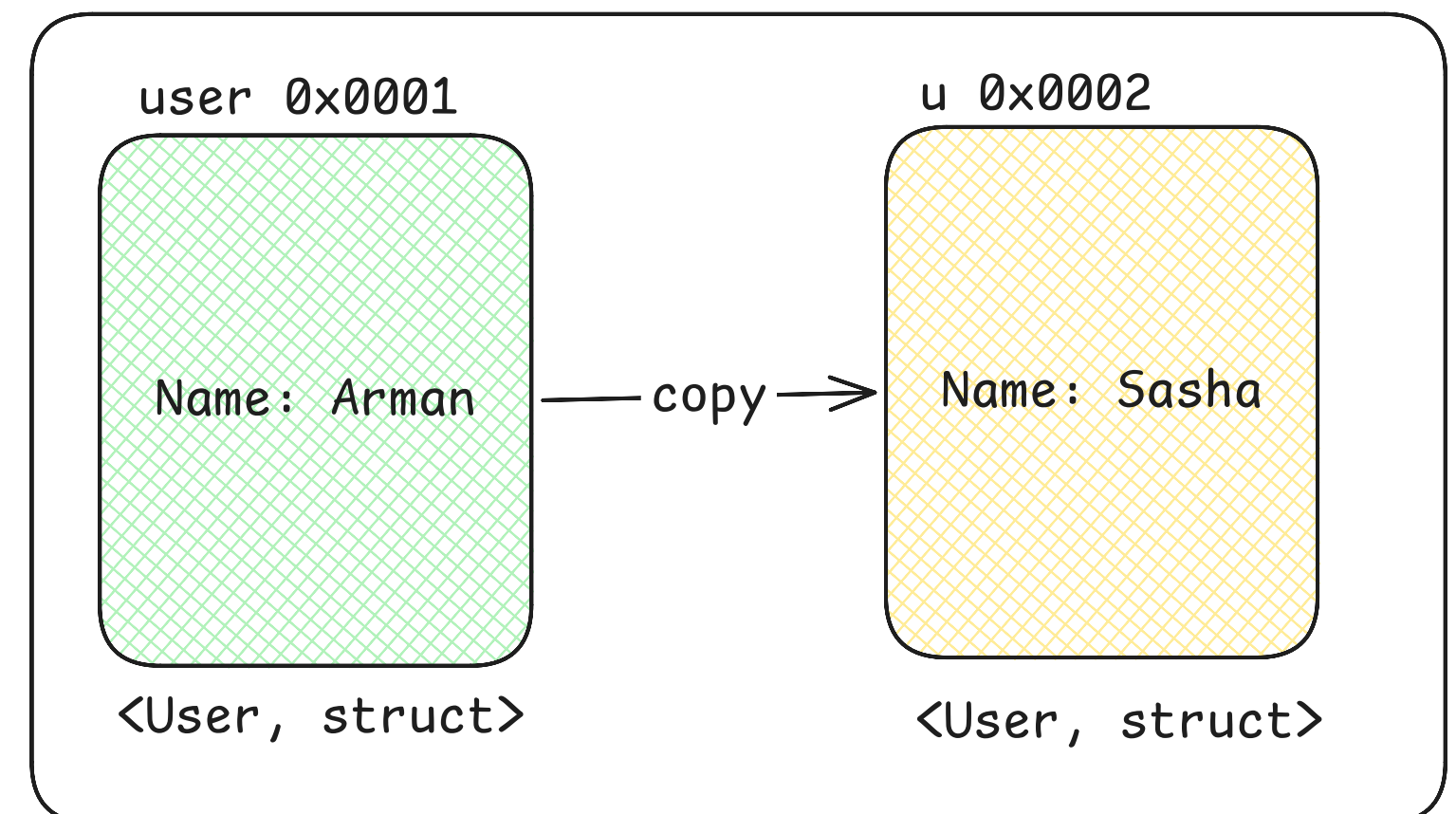
```
type User struct {
    ID int64
    Name string
    Email string
}

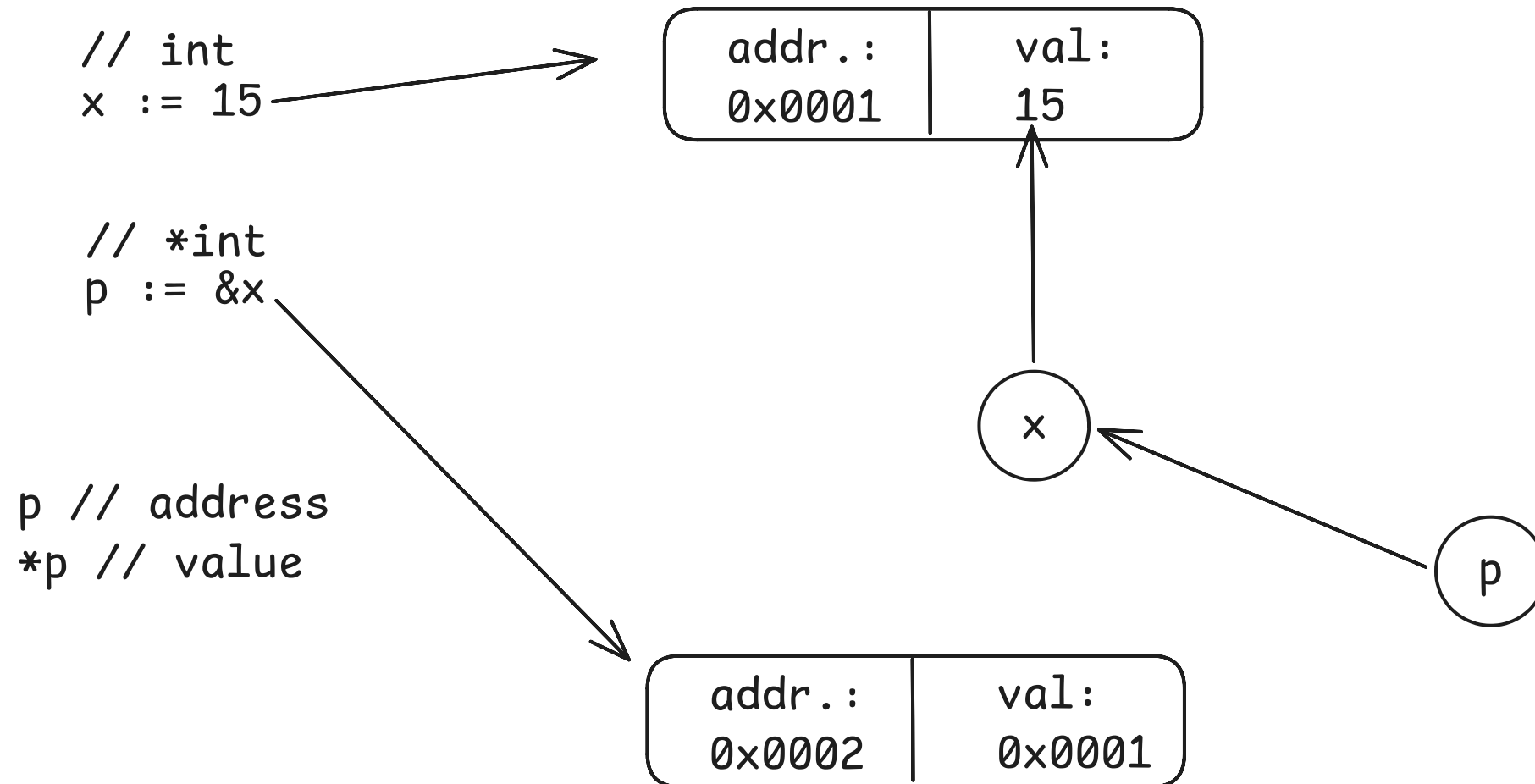
func rename(u User) {
    u.Name = "Sasha"
}

func main() {
    user := User{Name: "Arman"}

    rename(user)

    fmt.Println(user.Name) // Arman
}
```





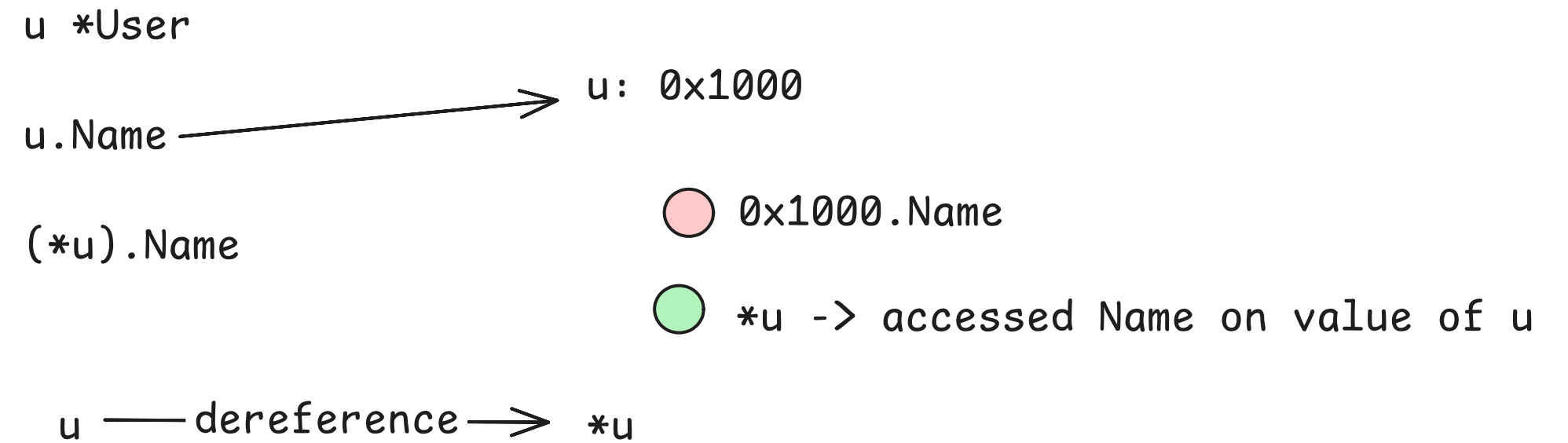
```
type User struct {
    ID int64
    Name string
    Email string
}

func rename(u *User) {
    u.Name = "Sasha"
}

func main() {
    user := User{Name: "Arman"}

    rename(&user)

    fmt.Println(user.Name) // Sasha
}
```



5

What is a method?

```
type Rectangle struct {  
    Width float64  
    Height float64  
}
```

```
func area(r Rectangle) float64 {  
    return r.Width * r.Height  
}
```

```
func main() {  
    r := Rectangle{  
        Width: 10.0,  
        Height: 15.0,  
    }  
  
    a := area(r)  
  
    fmt.Println(a) // 150  
}
```

receiver



6

Receivers

```
func (r Rectangle) area() float64 {  
    return r.Width * r.Height  
}
```

```
func main() {  
    r := Rectangle{  
        Width: 10.0,  
        Height: 15.0,  
    }  
  
    a := r.area()  
  
    fmt.Println(a) // 150  
}
```

